

Sprint 2

Marchini Claudio, Tomasi Cesare

Introduction

This sprint builds on top of the work of [sprint 1](#) and aims to integrate the last component of the system: the *cargorobot*.

Requirement Analysis

The requirement analysis has already been specified in the [sprint 0](#), this particular sprint does not require anything else to be specified in this section.

Problem Analysis

Using the previous [requirement analysis](#) as source, we will briefly summarize here the behaviour of the **cargorobot** actor:

1. It moves from the *HOME* to the *IOPORT*
2. It takes the container from *IOPORT* to the *slot-5*
3. Waits for the marking to be done
4. Moves the container from the *slot-5* to the assigned slot
5. It goes back to the *HOME*

As described in the [sprint 0](#) we have a few implementations of an already existing *cargorobot* to choose from:

- [VirtualRobot26](#)

- RobotObj26
- RobotService26
- RobotSmart26

	VirtualRobot26	RobotObj26	RobotService26	RobotSma
Language	Java	Java	Qak	Qak
Protocol	HTTP/WS	Java interface	Dispatch/ Request	Dispatch/ Request
Syntax	cril	cril	aril	aril
Synchronism	Sync/Async	Sync	Async	Async
Pathfinding	None	None	None	A*

The implementation that matches more closely our needs is **RobotSmart26** because:

- We need to move the robot between places that are not always fixed and known beforehand, so a pathfinding service is needed in our use case.
- The system need to stay reactive during the robot's movement, so an asynchronous implementation is needed.
- The robot uses a map system that closely resembles our hold implementation.

The code of *RobotSmart26* is available on the [repository](#), since our project shouldn't modify the code, it will be compiled as is and the rest of the project will interact only by the *qak* messages:

```
Request buildPlan : buildPlan(PX,PY,TX,TY)
Reply buildPlanDone : buildPlanDone( PLAN ) for buildPlan

Request moverobot : moverobot(TARGETX, TARGETY,STEPTIME)
Reply moverobotdone : moverobotok(ARG) for moverobot
Reply moverobotfailed : moverobotfailed(PLANDONE, PLANTODO) for moverobot

Dispatch noplan : noplan(X)
Dispatch setplanbuilddelay : value(V)

Request doplan : doplan(PLAN, STEPTIME)
Reply doplandone : doplandone(ARG) for doplan
Reply doplanfailed : doplanfailed(PLANTODO) for doplan

Request step : step(TIME)
Reply stepdone : stepdone(V) for step
Reply stepfailed : stepfailed(DURATION, CAUSE) for step

Dispatch move : move(M)
Dispatch setrobotstate : setpos(X,Y,D)

Request setdirection : dir(D)
Reply setdirectiondone : pos(PX,PY) for setdirection

Request tuneAtHome : tuneAtHome(X)
Reply tuneDone : tuneDone(X) for tuneAtHome

Request getrobotstate : getrobotstate(ARG)
Reply robotstate : robotstate(POS,DIR) for getrobotstate
```

Test Plans

By the end of the sprint every component of the system has implemented in its entirety, so the user can fully test each part and see if it meets the requirements. As in the sprint 1 the tests are end-to-end and no unit tests are required. To see if the system works, a virtual environment containing a simulation of the robot and the hold is available as a web page.

Deployment

The system will need two more containers:

- The cargorobot subsystem
- A virtual environment to test the system

As said before, because the *cargorobot's* code should not be touched we decided to distribute the precompiled code in a single *tar* file. The system will also need a couple files to load the hold map and some initialization parameters

```
FROM eclipse-temurin:17.0.5_8-jre-focal AS builder

ADD ./external/robotsmart26/robotsmart26-1.0.tar /

WORKDIR /robotsmart26-1.0/bin

COPY ./*.pl ./
COPY ./external/robotsmart26/basicrobotParams.json ./
COPY ./*.bin ./
COPY ./external/robotsmart26/tf25map.txt ./

CMD ["bash", "robotsmart26"]
```

The wenv instead will simply be pulled from the docker registry

```
# Other services from sprint 1
# ...
wenv:
  container_name: wenv
  image: docker.io/natbodocker/virtualrobotdisi26:1.0
  ports:
    - 8090:8090
    - 8091:8091/tcp
    - 8091:8091/udp
  restart: unless-stopped

robotoutgui25:
  container_name: robotoutgui25
  image: docker.io/natbodocker/robotoutgui25:1.0
  ports:
    - 8085:8085/tcp
  restart: unless-stopped

robotsmart26:
  container_name: robotsmart26
  build:
    dockerfile: ./docker/Dockerfile.robotsmart26
  ports:
    - "8020:8020/tcp"
    - "8020:8020/udp"
  environment:
    - VIRTUAL_ENV=wenv
    - MQTTBROKER=mosquitto
  depends_on:
    - wenv
  restart: unless-stopped
```

**Marchini Claudio**

claudio.marchini@studio.unibo.it

**Tomasi Cesare**

cesare.tomasi@studio.unibo.it

 SignedSnow0/CargoService